Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 1: Introducción al uso de Linux

Historia, distribuciones y línea de comandos

Índice

1. Introducción **2**

2. Fundamentos de Linux **3**
2.1. Historia de Linux . . . 3
2.2. Distribuciones de Linux . . . 3
2.3. El shell . . . 4
2.4. Sistema de archivos y rutas . . . 4
2.5. Comandos básicos . . . 4
2.5.1. Navegación . . . 4
2.5.2. Gestión de archivos y directorios . . . 5
2.5.3. Permisos . . . 5
2.5.4. Búsqueda y procesamiento de texto . . . 5
2.5.5. Edición de texto con `sed` . . . 6
2.5.6. Sustitución de comandos (`$()`) . . . 7
2.5.7. Procesos . . . 8
2.5.8. Depuración de llamadas al sistema . . . 8
2.5.9. Espacio en disco . . . 9
2.5.10. Variables de entorno . . . 9
2.5.11. Redirección y *pipes* . . . 9
2.5.12. Compresión y archivado . . . 10
2.5.13. Red y descarga de archivos . . . 10
2.5.14. Conexión remota y transferencia de archivos . . . 10
2.5.15. Sesiones persistentes con `tmux`/`screen` . . . 11
2.5.16. Información del sistema y dispositivos . . . 12
2.5.17. Creación de imágenes y copia de datos con `dd` . . . 12
2.6. Control de versiones con Git . . . 13

3. Evaluación **14**
3.1. Preguntas teóricas . . . 14
3.2. Ejercicios prácticos . . . 14
3.3. Entregable . . . 15

1. Introducción

El desarrollo de sistemas empotrados se realiza, en su gran mayoría, sobre herramientas que corren en **Linux**: desde las estaciones de trabajo de desarrollo hasta el sistema operativo que finalmente se ejecuta sobre el hardware objetivo (por ejemplo, distribuciones embebidas basadas en Yocto o Buildroot). Por esta razón, el manejo fluido de la línea de comandos de Linux es una habilidad indispensable para el resto del curso: los talleres posteriores (compilación con GCC, construcción con Make/Autotools/CMake, depuración con GDB, manejo de QEMU) asumen que el estudiante puede moverse por el sistema de archivos, manipular archivos y procesos, y automatizar tareas simples desde la terminal sin depender de un entorno gráfico.
Este taller presenta los fundamentos históricos y conceptuales de Linux, así como los comandos de uso más frecuente. Al finalizar, el estudiante aplicará estos conocimientos resolviendo una serie de ejercicios prácticos que combinan varios comandos entre sí.

2. Fundamentos de Linux

2.1. Historia de Linux

Linux nace en 1991, cuando el estudiante finlandés **Linus Torvalds** publicó el código fuente de un núcleo (*kernel*) de tipo Unix que había desarrollado como proyecto personal, con la intención de ejecutarlo en su propia computadora. Torvalds anunció el proyecto en un mensaje al grupo de noticias `comp.os.minix`, invitando a otros a colaborar; ese modelo de desarrollo abierto y distribuido es, hasta la fecha, una de las características que distingue a Linux.
El kernel de Linux, sin embargo, no constituye un sistema operativo completo por sí solo: necesita herramientas de compilación, bibliotecas del sistema, utilidades de línea de comandos, entre otros. Estas piezas ya existían gracias al **proyecto GNU** (*GNU's Not Unix*), iniciado por **Richard Stallman** en 1983 con el objetivo de construir un sistema operativo completo, libre y compatible con Unix. Para 1991 el proyecto GNU contaba con compilador (`gcc`), shell (`bash`), editor (`emacs`) y muchas otras utilidades, pero le faltaba un kernel propio maduro. La combinación del kernel de Linux con las herramientas de GNU dio origen a lo que técnicamente debería llamarse **GNU/Linux**, aunque en la práctica se le conoce simplemente como *Linux*.
Esta historia explica por qué Linux es, desde su origen, un sistema profundamente ligado a la filosofía del **software libre**: el código fuente del kernel y de la mayoría de sus herramientas está disponible para ser estudiado, modificado y redistribuido. Esa apertura es también la razón por la que Linux se convirtió en la base natural de la mayoría de los sistemas embebidos actuales (routers, electrodomésticos inteligentes, sistemas automotrices, servidores, superocomputadoras y la inmensa mayoría de dispositivos Android): un fabricante puede tomar el kernel, adaptarlo a su hardware específico y construir alrededor de él un sistema a la medida, sin pagar licencias ni depender de un único proveedor.

2.2. Distribuciones de Linux

Una **distribución** (o *distro*) es un paquete completo que combina el kernel de Linux con un conjunto coherente de herramientas y políticas para formar un sistema operativo usable. Una distribución típica incluye:

- El **kernel** de Linux, usualmente con una configuración y parches específicos.
- Un **gestor de paquetes** para instalar, actualizar y eliminar software (por ejemplo `apt` en Debian/Ubuntu, `dnf` en Fedora, `pacman` en Arch).
- Bibliotecas del sistema y herramientas de compilación (glibc, GCC, binutils).
- Un sistema de **inicio** (*init*), como `systemd`, encargado de arrancar servicios en el orden correcto.
- Opcionalmente, un entorno de escritorio (GNOME, KDE, XFCE) para uso interactivo con interfaz gráfica.

Existen decenas de distribuciones orientadas a propósitos distintos: Ubuntu y Debian priorizan estabilidad y facilidad de uso en escritorio y servidores; Fedora y Arch priorizan software reciente; distribuciones como **Raspberry Pi OS** están adaptadas a hardware específico. En el ámbito de **sistemas empotrados**, en lugar de instalar una distribución genérica, es común *construir* una distribución a la medida del hardware objetivo usando herramientas como **Yocto Project** o **Buildroot**, que generan una imagen mínima con únicamente el kernel y las utilidades necesarias para la aplicación (tema que se explora en talleres posteriores del curso).

2.3. El shell

El **shell** es el intérprete de comandos: un programa que lee las instrucciones que el usuario escribe en la terminal, las interpreta y solicita al sistema operativo que las ejecute. Es la interfaz de texto entre el usuario y el kernel. El shell más común en distribuciones Linux es **bash** (*Bourne Again SHell*), aunque existen alternativas como `zsh`, `fish` o `dash`.
Además de ejecutar comandos individuales, el shell es un lenguaje de *scripting* completo: permite encadenar comandos, usar variables, condicionales y ciclos para automatizar tareas repetitivas. Esta capacidad de automatización es una de las razones por las que el manejo de línea de comandos es preferible al uso de una interfaz gráfica en tareas de desarrollo e integración de sistemas empotrados, donde muchas veces ni siquiera se cuenta con entorno gráfico (por ejemplo, al conectarse por SSH a una tarjeta embebida).

2.4. Sistema de archivos y rutas

Linux organiza todos los archivos del sistema en un único árbol jerárquico que comienza en la raíz `/`, independientemente de cuántos discos o particiones físicas existan. Algunos directorios estándar (definidos por el *Filesystem Hierarchy Standard*) son:

Directorio	Contenido
`/bin`, `/usr/bin`	Ejecutables de comandos básicos del sistema.
`/etc`	Archivos de configuración del sistema.
`/home`	Directorios personales de cada usuario.
`/dev`	Archivos especiales que representan dispositivos de hardware.
`/proc`, `/sys`	Sistemas de archivos virtuales con información del kernel y los procesos en ejecución.
`/var`	Datos variables: logs, colas, cachés.

Para referirse a un archivo o directorio dentro de este árbol se utiliza una **ruta** (*path*), que puede ser de dos tipos:

- **Ruta absoluta**: describe la ubicación exacta de un archivo desde la raíz del sistema, por ejemplo `/home/usuario/taller1/datos.txt`. Siempre comienza con `/` y es independiente del directorio actual desde el que se ejecute el comando.
- **Ruta relativa**: describe la ubicación de un archivo respecto al **directorio actual** de trabajo, por ejemplo `taller1/datos.txt` o `../otro_directorio/`. Los símbolos especiales `.` (directorio actual), `..` (directorio padre) y `~` (directorio personal del usuario) se usan frecuentemente para construir rutas relativas.

2.5. Comandos básicos

A continuación se agrupan por categoría los comandos de uso más frecuente en la línea de comandos de Linux.

2.5.1. Navegación

`pwd`	Imprime el directorio de trabajo actual (*print working directory*).
`cd <ruta>`	Cambia el directorio actual.
`ls`	Lista el contenido de un directorio (`-l` formato detallado, `-a` incluye archivos ocultos, `-h` tamaños legibles).

2.5.2. Gestión de archivos y directorios

`mkdir <dir>`	Crea un directorio.
`touch <archivo>`	Crea un archivo vacío o actualiza su fecha de modificación.
`cp <origen> <destino>`	Copia archivos o directorios (`-r` para copiar directorios completos).
`mv <origen> <destino>`	Mueve o renombra archivos.
`rm <archivo>`	Elimina archivos (`-r` para directorios, `-f` sin confirmación).
`cat <archivo>`	Muestra el contenido completo de un archivo.

2.5.3. Permisos

Linux es un sistema **multiusuario**: cada archivo tiene un propietario, un grupo y un conjunto de permisos de lectura (r), escritura (w) y ejecución (x), definidos por separado para el propietario, el grupo y el resto de usuarios. Estos permisos se observan con `ls -l` y se modifican con:

`chmod <modo> <archivo>`	Cambia permisos, por ejemplo `chmod +x script.x` agrega permiso de ejecución.
`chown <usuario>:<grupo> <archivo>`	Cambia el propietario y/o grupo de un archivo.

2.5.4. Búsqueda y procesamiento de texto

`grep` y `find` son, junto con los *pipes*, las herramientas de búsqueda más usadas a diario: `grep` busca **contenido dentro de archivos**, mientras que `find` busca **archivos dentro del sistema de archivos** según su nombre, tipo, tamaño, fecha o permisos.

`grep <patrón> <archivo>`	Busca líneas que contienen un patrón de texto.
`grep -r <patrón> <directorio>`	Busca de forma recursiva en todos los archivos de un directorio; muy usado para localizar dónde se usa una función o variable dentro de un proyecto de código fuente.
`grep -n <patrón> <archivo>`	Muestra, junto a cada coincidencia, el número de línea donde aparece.
`grep -c, -i, -v`	`-c` cuenta coincidencias en vez de mostrarlas, `-i` ignora mayúsculas/minúsculas, `-v` invierte la búsqueda (muestra las líneas que **no** contienen el patrón).
`find <ruta> -name <patrón>`	Busca archivos por nombre dentro de un árbol de directorios (admite comodines, entre comillas).
`find <ruta> -type f/d`	Restringe la búsqueda a archivos (`f`) o directorios (`d`).
`find <ruta> -perm <modo>`	Busca archivos que tengan exactamente un conjunto de permisos dado, por ejemplo `find . -perm 777` localiza archivos con permisos totalmente abiertos.
`find <ruta> -mtime -n`	Busca archivos modificados en los últimos `n` días; útil para ubicar los archivos más recientes de un proyecto.
`find <ruta> -exec <comando> {} \;`	Ejecuta un comando sobre cada archivo encontrado, por ejemplo `find . -name "*.shexec chmod +x {} \;` otorga permiso de ejecución a todos los scripts `.sh` encontrados.
`wc`	Cuenta líneas (`-l`), palabras (`-w`) o caracteres (`-c`) de un archivo o de la entrada estándar.
`head/tail`	Muestran las primeras/últimas líneas de un archivo (`tail -f` sigue el archivo en tiempo real, útil para observar un log mientras se genera).

Por ejemplo, para encontrar dentro de un proyecto todos los archivos de permisos peligrosos (escritura habilitada para cualquier usuario del sistema) y corregirlos:

```
find . -type f -perm -o=w -exec chmod o-w {} \;
```

2.5.5. Edición de texto con `sed`

`sed` (*stream editor*) es un editor de texto no interactivo: en lugar de abrir un archivo para editarlo manualmente, se le indica una serie de instrucciones de transformación que aplica línea por línea, tomando el texto de un archivo o de la entrada estándar. Es especialmente útil para automatizar cambios repetitivos dentro de scripts (por ejemplo, modificar un archivo de configuración o un `Makefile` como parte de un proceso de construcción).

`sed 's/viejo/nuevo/' archivo`	Sustituye la **primera** aparición de `viejo` por `nuevo` en cada línea, e imprime el resultado en la salida estándar (el archivo original no se modifica).
`sed 's/viejo/nuevo/g' archivo`	Igual, pero el sufijo `g` (*global*) sustituye **todas** las apariciones en cada línea, no solo la primera.
`sed -i 's/viejo/nuevo/g' archivo`	La opción `-i` (*in-place*) escribe los cambios directamente sobre el archivo, en vez de solo mostrarlos por pantalla.
`sed -n '2,5p' archivo`	Imprime únicamente las líneas 2 a 5 del archivo (`-n` suprime la impresión automática de todas las líneas; `p` indica cuáles imprimir explícitamente).
`sed '/patrón/d' archivo`	Elimina (`d`) todas las líneas que contienen `patrón`.

Por ejemplo, para cambiar en todos los archivos de código fuente `.c` de un directorio la macro `DEBUG_LEVEL` de `1` a `0` (un caso común al preparar una versión de producción del firmware):

```
sed -i 's/DEBUG_LEVEL␣1/DEBUG_LEVEL␣0/g' *.c
```

2.5.6. Sustitución de comandos (`$()`)

La **sustitución de comandos** permite ejecutar un comando y usar su salida estándar como si se hubiera escrito directamente en su lugar, dentro de otro comando. La sintaxis moderna es `$(comando)` (existe también la sintaxis histórica con comillas invertidas `` `comando` ``, pero `$()` es preferible porque admite anidar sustituciones sin ambigüedad y es más legible).

```
# El shell ejecuta "date" primero, y sustituye su salida en el lugar
# de $(date) antes de ejecutar "echo"
echo $(date)

# Puede combinarse con texto literal dentro de la misma linea
echo "Hoy␣es␣$(date␣+%Y-%m-%d)␣y␣el␣usuario␣activo␣es␣$(whoami)"

# Es comun guardar el resultado en una variable para reutilizarlo
FECHA=$(date +%Y-%m-%d)
echo "Respaldo␣generado␣el␣$FECHA"
```

Esta construcción es especialmente útil para pasar el resultado de un comando directamente como **argumento** de otro, sin pasos intermedios ni copiar valores manualmente; por ejemplo, más adelante en esta sección se usa `kill -9 $(pgrep firmware_sim)` para obtener el *PID* de un proceso por nombre y terminarlo en un solo comando.

2.5.7. Procesos

`ps`	Lista procesos en ejecución (`ps aux` para ver todos los procesos del sistema con detalle).
`top`	Muestra en tiempo real el uso de CPU y memoria por proceso, actualizándose periódicamente.
`htop`	Versión interactiva y a color de `top`, con soporte de mouse y filtrado; no viene instalado por defecto en todas las distribuciones.
`kill <pid>`	Envía una señal (por defecto `SIGTERM`) a un proceso para solicitar su finalización de forma ordenada, identificándolo por su *PID*.
`kill -9 <pid>`	Envía la señal `SIGKILL`, que fuerza la terminación inmediata del proceso sin darle oportunidad de liberar recursos ordenadamente. Debe usarse solo cuando `kill` (sin `-9`) no logra terminar el proceso, ya que un proceso puede ignorar `SIGTERM` pero nunca `SIGKILL`.
`pgrep <nombre>`	Busca procesos por nombre (o por otros criterios) y muestra su *PID*, sin necesidad de filtrar la salida de `ps` manualmente.
`pkill <nombre>`	Envía una señal directamente a todos los procesos que coincidan con un nombre; equivale a combinar `pgrep` con `kill`.
`killall <nombre>`	Envía una señal a todos los procesos que coincidan con un nombre, sin necesidad de conocer su *PID*.

Por ejemplo, para localizar el *PID* de un proceso llamado `firmware_sim` que dejó de responder, y forzar su terminación:

```
pgrep firmware_sim
kill -9 $(pgrep firmware_sim)
```

Esto es equivalente a usar directamente `pkill -9 firmware_sim`, que combina ambos pasos en un solo comando.

2.5.8. Depuración de llamadas al sistema

Cuando un programa falla y no se dispone de su código fuente (o el problema no es evidente revisándolo), es útil observar **qué le pide al sistema operativo** mientras se ejecuta: qué archivos abre, qué señales recibe, qué funciones de biblioteca invoca.

`strace <programa>`	Muestra, en tiempo real, cada **llamada al sistema** (*syscall*) que realiza el programa: apertura de archivos, lectura/escritura, señales, memoria, etc.
`ltrace <programa>`	De forma similar a `strace`, muestra las llamadas a **funciones de biblioteca** (por ejemplo `malloc`, `printf`) que realiza el programa.

Por ejemplo, para diagnosticar por qué un programa no logra abrir un archivo de configuración:

```
strace ./programa 2>&1 | grep open
```

2.5.9. Espacio en disco

En sistemas empotrados el almacenamiento suele ser limitado (memoria eMMC o tarjetas SD de pocos gigabytes), por lo que vigilar el espacio disponible es una tarea frecuente:

`df -h`	Muestra el espacio usado y disponible de cada sistema de archivos montado, en formato legible (*human-readable*).
`du -sh <directorio>`	Muestra el tamaño total ocupado por un directorio (`-s` resume en un solo total, en vez de detallar cada subdirectorio).

2.5.10. Variables de entorno

Las **variables de entorno** son valores con nombre que el shell pone a disposición de los programas que ejecuta (por ejemplo `PATH`, que indica en qué directorios buscar los ejecutables, o `HOME`, el directorio personal del usuario). El comando `env` imprime todas las variables de entorno actualmente definidas; `export VAR=valor` define una nueva variable (o modifica una existente) y la hace visible para los procesos hijos; `echo $VAR` imprime el valor de una variable específica.

`env`	Imprime todas las variables de entorno definidas en la sesión actual.
`env | grep VAR`	Filtra la salida de `env` para mostrar únicamente la(s) variable(s) cuyo nombre contiene `VAR`.
`export VAR=valor`	Define o modifica una variable de entorno y la exporta a los procesos hijos del shell actual.
`echo $VAR`	Imprime el valor actual de la variable `VAR`.

2.5.11. Redirección y *pipes*

Por defecto, todo programa en Linux tiene tres flujos de datos: entrada estándar (*stdin*), salida estándar (*stdout*) y salida de error (*stderr*). El shell permite redirigir estos flujos:

`comando > archivo`	Redirige la salida estándar hacia un archivo, sobrescribiéndolo.
`comando >> archivo`	Redirige la salida estándar agregándola al final del archivo.
`comando < archivo`	Usa el contenido de un archivo como entrada estándar.
`comando1 | comando2`	**Pipe**: conecta la salida estándar de `comando1` con la entrada estándar de `comando2`.

Los *pipes* son una de las herramientas más poderosas de la línea de comandos porque permiten combinar utilidades simples (que hacen una sola cosa bien) en flujos de procesamiento complejos, sin necesidad de escribir un programa nuevo. Por ejemplo, para saber cuántas cuentas de usuario del sistema tienen `bash` configurado como su *shell* por defecto:

```
cat /etc/passwd | grep "/bin/bash" | wc -l
```

Aquí `cat` envía el contenido de `/etc/passwd` (el archivo que registra cada cuenta de usuario del sistema) a `grep`, que filtra únicamente las líneas que terminan referenciando `/bin/bash` como shell, y `wc -l` cuenta cuántas líneas recibió.

2.5.12. Compresión y archivado

`tar -cvf archivo.tar <archivos>`	Empaqueta varios archivos en un único archivo `.tar` (`-x` para extraer, `-z` para comprimir/descomprimir con gzip).
`md5sum <archivo>`	Calcula la suma de verificación MD5 de un archivo, útil para comprobar su integridad.

2.5.13. Red y descarga de archivos

`wget <url>`	Descarga un archivo desde una URL.
`curl <url>`	Transfiere datos desde o hacia una URL; más flexible que `wget` para interactuar con APIs.

2.5.14. Conexión remota y transferencia de archivos

En sistemas empotrados es común no contar con teclado ni pantalla conectados directamente a la tarjeta: el desarrollo se hace desde otra computadora, conectándose remotamente a través de la red. La herramienta estándar para esto es **SSH** (*Secure Shell*), que permite abrir una sesión de línea de comandos en un equipo remoto de forma cifrada:

`ssh usuario@host`	Abre una sesión de shell remota en `host`, autenticándose como `usuario` (por contraseña o llave). Si se omite `usuario`, se usa el usuario actual.
`ssh -p <puerto> usuario@host`	Igual, pero especificando un puerto distinto al 22 (por defecto).
`ssh usuario@host <comando>`	Ejecuta un único comando en el equipo remoto y retorna, sin abrir una sesión interactiva.
`ssh-copy-id usuario@host`	Copia la llave pública propia al equipo remoto, para poder autenticarse sin contraseña en conexiones futuras.

Para transferir archivos hacia o desde un equipo remoto, sin necesidad de un servidor FTP, se usan herramientas que reutilizan el mismo canal cifrado de SSH:

`scp archivo.txt usuario@host:ruta/`	**Copia** un archivo local hacia una ruta remota (`-r` para copiar directorios completos).
`scp usuario@host:archivo.txt .`	Copia un archivo remoto hacia el directorio actual local. Nótese que el lado remoto siempre se identifica con `usuario@host:ruta`.
`rsync -avz origen/ usuario@host:destino/`	**Sincroniza** un directorio local con uno remoto, transfiriendo únicamente los archivos nuevos o modificados; más eficiente que `scp` para transferencias repetidas o de gran tamaño (`-a` preserva permisos y enlaces, `-v` modo detallado, `-z` comprime durante la transferencia).

Por ejemplo, para conectarse a una tarjeta embebida en la red local y copiar hacia ella un binario recién compilado:

```
# Conectarse por SSH a la tarjeta
ssh usuario@192.168.1.50

# Desde la maquina de desarrollo: copiar un archivo hacia la tarjeta
scp firmware.bin usuario@192.168.1.50:/home/usuario/

# Copiar de vuelta un archivo de log generado en la tarjeta
scp usuario@192.168.1.50:/home/usuario/log.txt .
```

2.5.15. Sesiones persistentes con `tmux`/`screen`

Al trabajar por SSH sobre una tarjeta embebida (o cualquier servidor remoto), cerrar la conexión —ya sea intencionalmente o por un corte de red— termina también todos los procesos que se estaban ejecutando en esa sesión de shell, incluyendo compilaciones largas o programas que deben seguir corriendo. Un **multiplexor de terminal** como `tmux` o `screen` resuelve este problema: crea una sesión de shell independiente de la conexión SSH que la originó, la cual puede **desconectarse** (*detach*) sin terminar los procesos que contiene, y **reconectarse** (*attach*) más tarde —incluso desde otra máquina— retomando exactamente donde quedó.

`tmux new -s <nombre>`	Crea una nueva sesión con un nombre identificable (sin `-s <nombre>`, la sesión recibe un número).
`tmux ls`	Lista las sesiones de `tmux` activas en el sistema, estén o no conectadas a una terminal en ese momento.
`tmux attach -t <nombre>`	Reconecta la terminal actual a una sesión existente, dejando visible todo lo que ocurrió mientras estuvo desconectada.
`Ctrl+b` luego `d`	**Detach**: se desconecta de la sesión actual sin terminarla; los procesos dentro de ella siguen ejecutándose en segundo plano.
`Ctrl+b` luego `c`	Crea una nueva ventana dentro de la misma sesión (`Ctrl+b <n>` cambia a la ventana número `n`).
`Ctrl+b` luego `%` / `"`	Divide la ventana actual en paneles, vertical u horizontalmente.
`tmux kill-session -t <nombre>`	Termina una sesión y todos los procesos que contiene.

`screen` es una herramienta más antigua con el mismo propósito; los comandos equivalentes son `screen -S <nombre>` (crear), `screen -ls` (listar), `screen -r <nombre>` (reconectar) y `Ctrl+a` luego `d` (*detach*). En la práctica `tmux` es más común en distribuciones recientes, pero `screen` suele estar preinstalado incluso en imágenes mínimas de sistemas empotrados.
Por ejemplo, para lanzar una compilación larga en una tarjeta remota y poder desconectarse con seguridad mientras se completa:

```
ssh usuario@192.168.1.50
tmux new -s build
make -j4                # compilacion larga
# Ctrl+b, luego d        -> se desconecta de la sesion sin detenerla
exit                     # se puede cerrar la conexion SSH con seguridad

# Mas tarde, o desde otra maquina, para revisar el resultado:
ssh usuario@192.168.1.50
tmux attach -t build
```

2.5.16. Información del sistema y dispositivos

`uname -r`	Muestra la versión del kernel de Linux en ejecución.
`whoami`	Muestra el nombre del usuario actual.
`dmesg`	Muestra el *buffer* de mensajes del kernel, incluyendo eventos de conexión de dispositivos.
`lsusb`	Lista los dispositivos USB conectados al sistema, junto con su identificador de *Vendor* y de producto.
`lsblk`	Lista los dispositivos de bloque (discos, tarjetas SD, memorias USB) y sus particiones, en forma de árbol.
`lspci`	Lista los dispositivos conectados al bus PCI/PCIe (por ejemplo, controladores de red o de video).
`journalctl`	Consulta el registro (*log*) centralizado de `systemd`, que incluye tanto mensajes del kernel como de los servicios del sistema (`journalctl -f` sigue el registro en tiempo real; `journalctl -u <servicio>` filtra por un servicio específico).

2.5.17. Creación de imágenes y copia de datos con `dd`

`dd` copia datos **a nivel de bytes** entre un origen y un destino, sin interpretar su contenido (a diferencia de `cp`, que opera sobre archivos dentro de un sistema de archivos). Esto lo hace la herramienta estándar para crear archivos de un tamaño exacto, o para leer y escribir **dispositivos de bloque completos** (tarjetas SD, discos, memorias USB) como si fueran un único archivo grande —una operación muy común en sistemas empotrados al grabar una imagen de sistema operativo en una tarjeta SD (por ejemplo, para Raspberry Pi).

`if=<archivo>`	Archivo o dispositivo de **entrada** (*input file*).
`of=<archivo>`	Archivo o dispositivo de **salida** (*output file*).
`bs=<tamaño>`	Tamaño de cada bloque copiado (por ejemplo `1K`, `4M`); bloques más grandes suelen copiar más rápido, a costa de mayor uso de memoria.
`count=<n>`	Número de bloques a copiar; sin este parámetro, `dd` copia hasta el final del archivo de entrada.
`status=progress`	Muestra el progreso de la copia en tiempo real, útil en copias grandes que tardan varios minutos.

Por ejemplo, para crear un archivo en blanco (relleno de ceros) de exactamente 1 KB, leyendo desde el dispositivo especial `/dev/zero` (que produce un flujo infinito de bytes cero):

```
dd if=/dev/zero of=empty.img bs=1K count=1
```

Y para grabar una imagen de sistema operativo descargada (`raspios.img`) en una tarjeta SD, previamente identificada con `lsblk` (**es indispensable verificar el dispositivo correcto antes de continuar**: `dd` sobrescribe sin pedir confirmación, y un error en `of=` puede destruir el disco duro del propio computador en vez de la tarjeta SD):

```
lsblk
sudo dd if=raspios.img of=/dev/sdX bs=4M status=progress
sync
```

El comando `sync` al final asegura que todos los datos en caché se escriban físicamente en el dispositivo antes de retirarlo.

2.6. Control de versiones con Git

Git es un sistema de control de versiones distribuido: registra el historial de cambios de un proyecto en forma de *commits*, permite trabajar en líneas de desarrollo paralelas (*branches*) y sincronizar el trabajo entre distintas copias del repositorio (locales o remotas, como GitHub o GitLab). Aunque no es parte del sistema operativo Linux en sí, es una herramienta que se usa a diario junto con la línea de comandos en cualquier proyecto de software o firmware, incluyendo los de este curso.

`git init`	Convierte el directorio actual en un nuevo repositorio Git (crea el subdirectorio oculto `.git`).
`git clone <url>`	Crea una copia local completa de un repositorio remoto, incluyendo todo su historial.
`git status`	Muestra el estado actual: archivos modificados, sin seguimiento o listos para *commit*.
`git add <archivo>`	Agrega los cambios de un archivo al *staging area*, marcándolos para el próximo *commit* (`git add .` agrega todos los cambios del directorio actual).
`git commit -m "mensaje"`	Registra en el historial los cambios que están en el *staging area*, junto con un mensaje descriptivo.
`git log`	Muestra el historial de *commits* (`--oneline` para un resumen de una línea por *commit*).
`git diff`	Muestra las diferencias línea por línea entre el contenido actual de los archivos y la última versión registrada.
`git branch <nombre>`	Crea una nueva rama; sin argumentos, lista las ramas existentes.
`git checkout <rama>`	Cambia a otra rama (o restaura archivos a una versión anterior).
`git merge <rama>`	Combina los cambios de otra rama dentro de la rama actual.
`git remote add <nombre> <url>`	Registra un repositorio remoto bajo un nombre corto (por convención, `origin`).
`git push`/`git pull`	Envía *commits* locales al repositorio remoto, o trae y combina los cambios remotos en el repositorio local, respectivamente.

Por ejemplo, el flujo típico para registrar un cambio y subirlo a un repositorio remoto ya configurado es:

```
git status
git add archivo_modificado.c
git commit -m "Corrige␣lectura␣de␣registro␣de␣estado␣del␣sensor"
git push origin main
```

3. Evaluación

3.1. Preguntas teóricas

1. ¿Qué es una distribución Linux y qué puede incluir?
2. ¿Qué es un *shell*?
3. ¿Cuál es la diferencia entre una ruta relativa y una ruta absoluta a un archivo?
4. ¿Qué información brinda el comando `env`?
5. ¿Para qué son útiles los *pipes* de UNIX? Muestre un ejemplo de su utilidad.

3.2. Ejercicios prácticos

1. Mediante el uso únicamente de línea de comandos (no utilice ningún editor de texto) genere un archivo llamado `Ejercicio1.txt` que contenga la siguiente información:

   - Encabezado:
     - Nombre (*su nombre completo*)
     - Carné (*su número de carné*)
   - Solución del ejercicio:
     - Fecha actual del sistema
     - Usuario: (*nombre de usuario en el sistema, no debe escribirlo explícitamente*)
     - Versión de kernel de Linux
     - Lista de procesos del usuario actual

   a) Presente los comandos necesarios para generar el archivo descrito anteriormente.
   b) Muestre el contenido final del archivo.

2. Dentro del directorio de esta evaluación, utilice el comando `mkdir` para crear un nuevo directorio llamado `Ej2`. Muestre la rutina de comandos necesarios para:

   - Generar un nuevo archivo (`Ejercicio2.txt`) que contenga la información de checksum MD5 del archivo `Ejercicio1.txt`.
   - Realizar un archivamiento (`Ejercicios.tar`) a partir de ambos archivos (`Ejercicio1.txt` y `Ejercicio2.txt`).
   - Copiar el archivo `Ejercicios.tar` en el directorio recién creado `Ej2`.
   - Ingresar al directorio `Ej2` y extraer allí el archivo `Ejercicios.tar`.

3. Dentro de un directorio llamado `Ej3`, muestre la rutina de comandos necesarios para:

   - Crear una imagen de archivo en blanco (`empty.img`), con un tamaño de 1KB.
   - Cambiar permisos de archivo para convertirlo en ejecutable (permiso de ejecución).
   - Escribir (por comandos) al archivo `empty.img` las siguientes 5 líneas:

     ```
     clear
     wget http://www.bolis.com/onyx/random/stuff/sounds/murray/murrays.wav
     aplay murrays.wav
     mv murrays.wav .murrays.wav
     echo Aquí no ha pasado nada...
     ```

- Renombrar el archivo `empty.img` a `script.x`.
- Ejecutar `script.x` (¿Qué debe suceder en este punto?).

4. En el siguiente enlace se encuentra la obra *La Odisea* de Homero, completa: `https://archive.org/stream/laodisea00homeuoft/laodisea00homeuoft_djvu.txt` Muestre los comandos necesarios para descargar el archivo y luego mostrar en consola la cuenta (número total) de las líneas que incluyan la palabra *Zeus*.

5. Dentro de un directorio llamado `Ej5`, muestre un comando que pueda utilizar para conocer el identificador de *Vendor* y el identificador de producto de los dispositivos USB que han sido conectados al sistema recientemente, desde que se inició.

6. Dentro de un directorio llamado `Ej6`, cree una estructura de al menos 3 subdirectorios, cada uno con al menos 2 archivos de texto con extensión `.log` y contenido libre inventado por usted, donde **algunas líneas** (no todas) contengan la palabra `ERROR`. A partir de esa estructura, genere un único comando (o secuencia de comandos encadenados con *pipes* y redirección) que realice lo siguiente:

   - Busque, de forma recursiva dentro de `Ej6`, todos los archivos `.log`.
   - Para cada uno, cuente cuántas líneas contienen la palabra `ERROR`.
   - Ordene los archivos de mayor a menor cantidad de coincidencias.
   - Guarde ese listado ordenado en un nuevo archivo `reporte.txt`, dentro de `Ej6`.

   Una vez generado `reporte.txt`, cambie sus permisos de forma que **únicamente el propietario** pueda leerlo y escribirlo (sin ningún permiso para grupo u otros). Finalmente, comprima el directorio `Ej6` completo (incluyendo `reporte.txt`) en un único archivo `Ej6.tar.gz`. Documente cada comando utilizado y explique brevemente qué hace cada parte de la secuencia.

7. Dentro de un directorio llamado `Ej7`, inicialice un nuevo repositorio Git y realice la siguiente secuencia de operaciones, documentando el comando utilizado en cada paso:

   a) Cree un archivo `version.txt` con el contenido `v1`, y regístrelo en un primer *commit*.

   b) Cree una nueva rama llamada `feature`, cámbiese a ella, y modifique `version.txt` para que contenga `v2`. Registre este cambio en un segundo *commit*, únicamente dentro de la rama `feature`.

   c) Regrese a la rama principal (`main` o `master`) y verifique, con el contenido de `version.txt`, que el cambio de la rama `feature` no está presente allí.

   d) Combine (*merge*) la rama `feature` dentro de la rama principal.

   e) Muestre el historial de *commits* resultante en formato resumido (una línea por *commit*).

   Presente la secuencia completa de comandos utilizados, junto con la salida de `git log --oneline` al finalizar.

3.3. Entregable

- Archivo `Tut1_Nombre_completo.tar.gz` que contenga los archivos y directorios de los ejercicios, así como un documento escrito con la solución de las preguntas teóricas y de los ejercicios prácticos. Suba el archivo al TecDigital.

Material de referencia

- The Linux Documentation Project: `https://tldp.org/`
- GNU Project: `https://www.gnu.org/gnu/thegnuproject.html`
- Bash Reference Manual: `https://www.gnu.org/software/bash/manual/`
- Filesystem Hierarchy Standard: `https://refspecs.linuxfoundation.org/fhs.shtml`
- Aguilar Ulloa, M. *Tutorial 1: Introducción al uso de Linux*. Material del curso, ITCR.